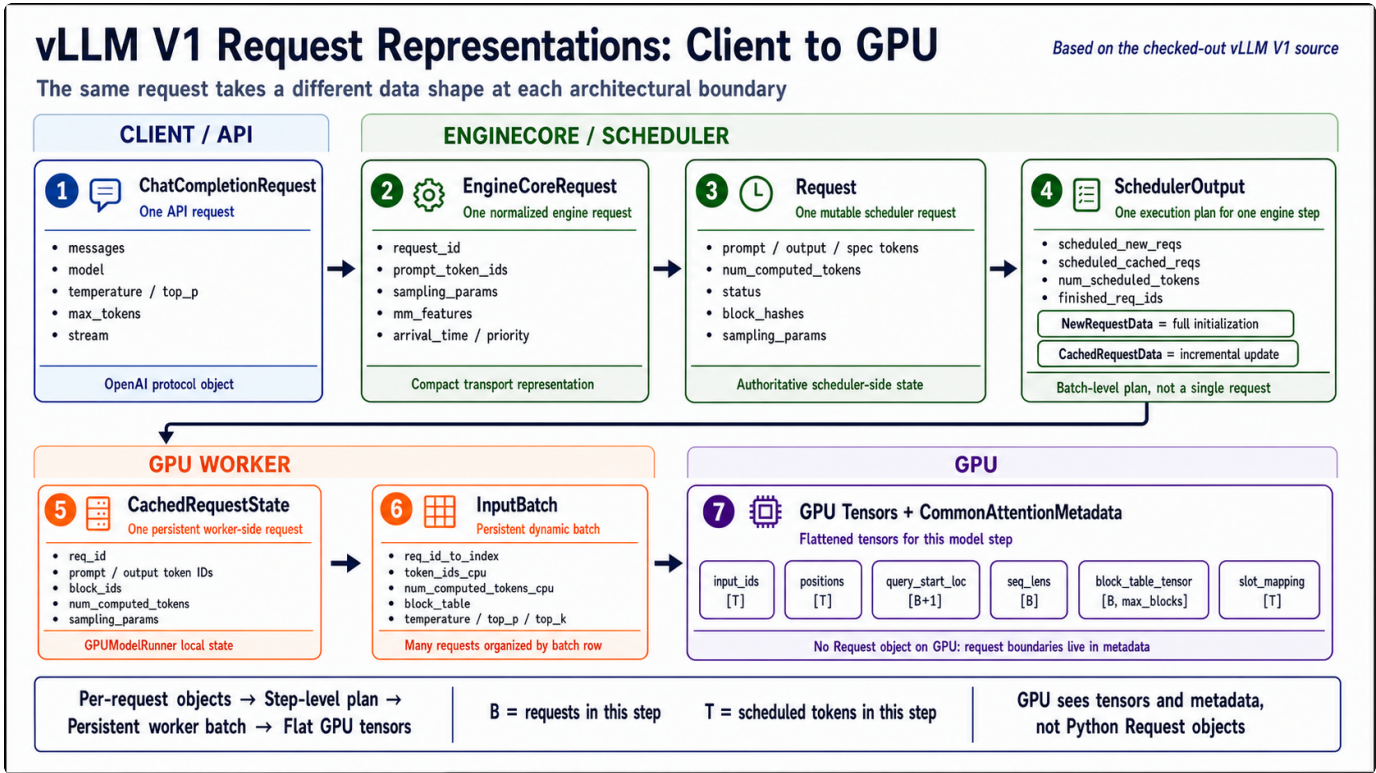


第4课 请求的数据形态：从 Client 到 GPU



1. Client: HTTP JSON

Client 发送的是协议数据，不是 vLLM 内部对象：

```
1 {
2   "model": "Qwen/Qwen2.5-1.5B-Instruct",
3   "messages": [
4     {"role": "user", "content": "Hello"}
5   ],
6   "temperature": 0.8,
7   "max_tokens": 128
8 }
```

它包含：



Plain Text |

- 1 业务输入
- 2 模型名称
- 3 生成参数
- 4 流式选项

此时还没有 token，也没有调度状态。

2. API Server: ChatCompletionRequest



Plain Text |

```
1 class ChatCompletionRequest:
2     model
3     messages
4     temperature
5     top_p
6     max_tokens
7     stream
```

这是 OpenAI API 层的请求实体：



Plain Text |

- 1 单位：单个用户请求
- 2 形态：Pydantic 业务对象
- 3 核心内容：messages + API 参数
- 4 作用：协议表达和参数校验

对于 offline inference，入口可能直接是：

[TextPrompt](#)

[TokensPrompt](#)

[EmbedsPrompt](#)

3. Engine 输入: EngineCoreRequest

▼ Plain Text |

```
1 class EngineCoreRequest:
2     request_id
3     prompt_token_ids
4     sampling_params
5     pooling_params
6     mm_features
7     arrival_time
8     priority
```

此时请求已经从业务语义变成推理语义：

▼ Plain Text |

```
1 messages      → prompt_token_ids
2 API 采样字段   → SamplingParams
3 图片/音频      → mm_features
```

结构特点：

▼ Plain Text |

```
1 单位：单个请求
2 形态：msgspec.Struct
3 核心内容：token、参数、多模态特征
4 位置：API Server 与 EngineCore 的边界
```

4. Scheduler: Request

这是最重要的请求实体：

```
1 class Request:
2     request_id
3     prompt_token_ids
4     output_token_ids
5     spec_token_ids
6
7     num_computed_tokens
8     status
9     block_hashes
10
11     sampling_params
12     pooling_params
```

结构特点：

```
1 单位：单个请求
2 形态：可变 Python 对象
3 所有者：Scheduler
4 作用：维护完整生命周期和计算进度
```

它是 Scheduler 侧的权威状态：

```
1 请求有哪些 token?
2 已经计算了多少?
3 当前是 WAITING 还是 RUNNING?
4 是否被抢占?
5 有哪些 block hash?
```

前面几种形态是“请求想做什么”，从这里开始变成“请求运行到哪里”。

5. Scheduler 输出：SchedulerOutput



Plain Text |

```
1 class SchedulerOutput:
2     scheduled_new_reqs
3     scheduled_cached_reqs
4     num_scheduled_tokens
5     total_num_scheduled_tokens
6     scheduled_spec_decode_tokens
7     finished_req_ids
```

它不是单个请求，而是：

Scheduler 为当前 engine step 生成的批量执行计划。

结构特点：



Plain Text |

```
1 单位：一整个调度 step
2 形态：批量计划
3 生产者：Scheduler
4 消费者：GPUModelRunner
```

内部有两种请求表示。

NewRequestData



Plain Text |

```
1 class NewRequestData:
2     req_id
3     prompt_token_ids
4     sampling_params
5     block_ids
6     num_computed_tokens
```

表示 Worker 第一次见到的请求，因此携带完整初始化信息。

CachedRequestData



Plain Text

```
1 class CachedRequestData:
2     req_ids
3     new_token_ids
4     new_block_ids
5     num_computed_tokens
6     num_output_tokens
```

表示 Worker 已经知道的请求，只携带本轮增量。

注意：

`CachedRequestData` 表示 Worker 缓存了请求状态，不表示 Prefix Cache 命中。

6. Worker: `CachedRequestState`



Plain Text

```
1 class CachedRequestState:
2     req_id
3     prompt_token_ids
4     output_token_ids
5     block_ids
6     num_computed_tokens
7     sampling_params
```

它是 Request 在 Worker 侧的表示：



Plain Text

```
1 单位：单个请求
2 形态：Worker 本地持久状态
3 所有者：GPUModelRunner
```

对比两个 Request：



Plain Text

```
1 Scheduler.Request
2     调度侧权威状态，决定谁运行
3
4 CachedRequestState
5     Worker 侧执行状态，准备模型输入
```

二者属于不同进程，不能共享同一个 Python 对象。

7. Worker 批次: **InputBatch**

InputBatch 不再表示单个请求，而是多个请求的持久动态批次：



Plain Text

```
1 class InputBatch:
2     req_id_to_index
3     token_ids_cpu
4     num_prompt_tokens
5     num_computed_tokens_cpu
6     block_table
7
8     temperature
9     top_p
10    top_k
11    frequency_penalties
```

可以把它理解成一张按 request index 组织的表：



Plain Text

```
1 row 0 → request A
2 row 1 → request B
3 row 2 → request C
```

结构特点：

- 1 单位: Worker 当前维护的动态批次
- 2 形态: CPU tensor、NumPy array、GPU tensor 的组合
- 3 作用: 把多个 CachedRequestState 聚合为批量数据

Continuous Batching 最终就体现在这个结构中:

- 1 请求完成 → 删除一行
- 2 新请求加入 → 增加一行
- 3 被抢占 → 暂时移除
- 4 每个 step 都可以改变批次成员

8. GPU: 模型输入 Tensor

进入 GPU 后, 不再存在一个完整的 `Request` 对象。

请求被打平成多个 tensor:

```

1  input_ids:           [T]
2  positions:           [T]
3  query_start_loc:     [B + 1]
4  seq_lens:            [B]
5  block_table_tensor:  [B, max_num_blocks]
6  slot_mapping:        [T]
```

其中:

- 1 B = 本轮请求数
- 2 T = 本轮所有请求调度的 token 总数

注意请求边界由元数据保存。例如:


```
1 input_ids = [A0, A1, B0, B1, B2, C0]
2
3 query_start_loc = [0, 2, 5, 6]
```

表示：

```
1 Request A → input_ids[0:2]
2 Request B → input_ids[2:5]
3 Request C → input_ids[5:6]
```

Attention 的公共元数据放在：[CommonAttentionMetadata](#)

```
1 class CommonAttentionMetadata:
2     query_start_loc
3     seq_lens
4     num_reqs
5     num_actual_tokens
6     block_table_tensor
7     slot_mapping
```

它解决两个问题：

```
1 query_start_loc / seq_lens
2 → 扁平 token 分别属于哪个请求？
3
4 block_table_tensor / slot_mapping
5 → KV Cache 应该从哪里读取、写到哪里？
```

总结

转换	核心目的
JSON → ChatCompletionRequest	协议校验

<code>ChatCompletionRequest</code> → 模型输入	Chat模板、分词、采样参数
模型输入 → <code>EngineCoreRequest</code>	统一不同请求入口
<code>EngineCoreRequest</code> → <code>Request</code>	建立运行时状态
<code>Request</code> → <code>SchedulerOutput</code>	生成单步执行计划
<code>SchedulerOutput</code> → <code>CachedRequestState</code>	同步Worker侧请求状态
<code>CachedRequestState</code> → <code>InputBatch</code>	聚合动态批次
<code>InputBatch</code> → GPU Tensor	构造本轮模型输入和KV地址

这条链路的核心规律是：

越靠上越接近用户语义，越靠下越接近批处理、内存地址和GPU计算。